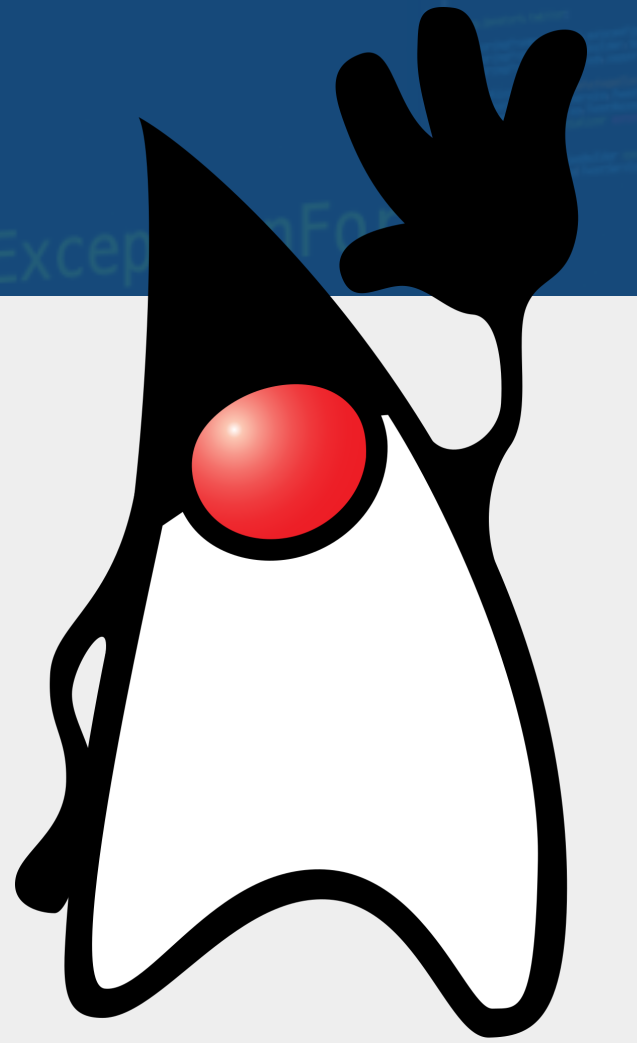


Java Programlama Dili

15. Bölüm

Nesnenin Davranışı

Akın Kaldıroğlu



Amaç



- Bu bölümde nesnenin davranışı (behavior) ele alınacaktır
- Dolayısıyla nesnenin metotları detaylandırılacaktır

Konular



- **Nesnenin Davranışı**
- **Metot**
 - Parametreler
 - Dönüş Tipi
 - Gerçekleştirme
 - Metotların Amaçları
 - get/set metotları
 - İmza ve Arayüz
- **Metotların Özellikleri**
 - Metotlarda Çoklu Kullanım
 - Değişen Sayıda Argümanlı Metotlar
 - Parametre Geçme
 - final Parametre
- **Ödevler**
- **Bundan Sonra?**

Nesnenin Davranışı



Davranış (Behavior)



- Nesneler davranır; belli işleri yapar ve sorumlulukları yerine getirir
- Yazılım nesnelerinin **davranışlarına (behavior)**, yerine getirdiği **sorumluluklar (responsibility)**, verdiği **hizmetler (service)** ya da aldığı **mesajlar (message)** olarak bakmak, işimizi kolaylaştırır

Mesajlaşma (Messaging)



- Nesneler davranışlarıyla, kendi durumları hakkında bilgi verebilir, durumlarını değiştirebilir veya bir işi yerine getirip bunun sonucunda da bir değer üretebilir
- Nesne sorumluluklarını yerine getirirken, özellikle 3. tür davranışlarda, muhtemelen başka nesnelerden yardım alır
- Nesnelerin bu şekilde birbirlerine hizmet vermesine **mesajlaşma (messaging)** denir

Metot Nedir?



- **Metot (method)**, nesnelerin **sorumluluğunu (responsibility)** yerine getiren bir programlama yapısıdır
- Davranış ya da sorumluluk daha soyut bir kavramken, metot, davranış ya da sorumluluğun bir programlama dilinde, örneğin Java'da, kodlanmış, gerçekleştirilmiş halidir
- Metotlar, bir nesnenin dışarıya verdiği **hizmeti (service)** ya da bir başka deyişle, bir nesnenin alabileceği **mesajları (message)** ifade eder

Fonksiyon ve Prosedür



- **Metot (method)** için farklı programlama kültürlerinde farklı isimler kullanılır:
 - Fonksiyon (function)
 - Prosedür (procedure)
 - Subroutine
 - Subprogram
- Bu farklı kullanımlar her zaman nesne bağlamında değildir

Metot



Metot - I



- Metotlar, sadece sınıf içinde tanımlanabilir
- Metotlar, içinde tanımlandığı sınıfın nesneleri üzerinden çağrılabilir
- Bir sınıfın tanımlamadığı bir metot, o sınıfın nesnelerinde çağrılmaz
- Bir sınıftan yaratılan bütün nesneler, o sınıfta tanımlanan metotları yerine getirir, yani hepsi aynı mesajları alır, aynı hizmetleri verir

Metot - II



- Bir sınıfta tanımlanan metotlara **üye metotlar (member methods)** denir
- Bir sınıfta istenildiği kadar metot tanımlanabilir

Metodun Bileşenleri - I



- Tipik olarak metotlar, dışarıdan malzeme olarak parametre alıp işler ve geriye, çağrıldıkları ortama bir değer döndürür
- Java'da bir metodun altı parçası vardır: İsim, parametre listesi, dönüş tipi, gerçekleştirme, fırlatılan sıra dışı durumlar ve niteleyiciler

Metodun Bileşenleri - II



- **İsim** (**name**): Anlamlı ve genelde emir kipinde bir isim
- **Parametreler** (**parameters**): Dışarıdan alabileceği değerlerin listesi
- **Dönüş tipi** (**return type**): Döndürdüğü değerın tipi
- **Gerçekleştirme** (**body, implementation**): Cümlelerden oluşan ve davranışı yerine getiren kısım

```
<modifier>* <return type> <name>(<Parameter>*) throws <exception>* {  
    <statement>*  
}
```

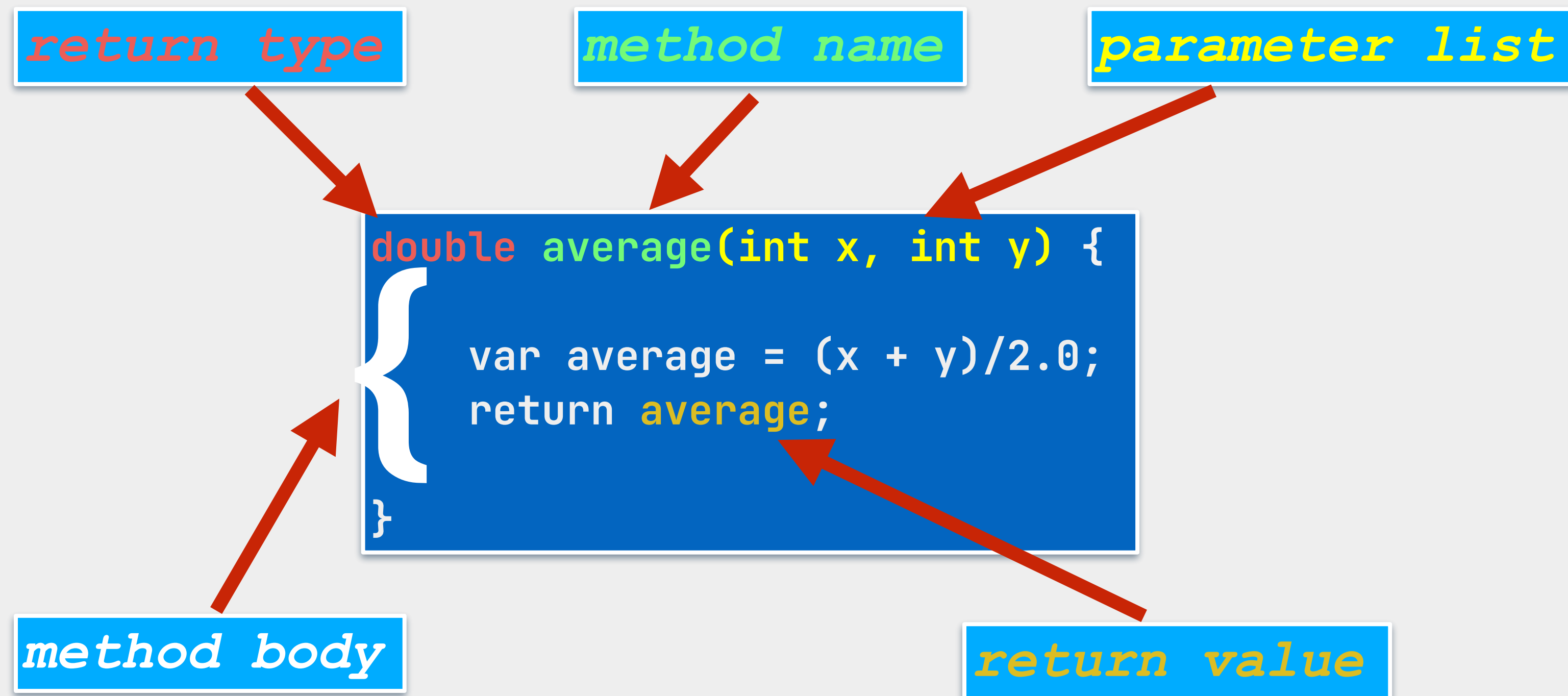
```
<niteleyici>* <dönüş tipi> <isim>(<Parametre>*) throws <exception>* {  
    <cümle>*  
}
```

Metodun Bileşenleri - II



- **Niteleyiciler (modifiers)**: Farklı amaçlar için değişik niteleyici anahtar kelimeler kullanılabilir
- **Sıra dışı durumlar (exceptions)**: Fırlattığı sıra dışı durumlar
 - Bir metod sıfır ya da daha fazla sıra dışı durum fırlatabilir
 - Metod tanımında, kodunda fırlattığı tüm sıra dışı durumlar listelenir
 - Sıra dışı durumlar daha sonra ele alınacaktır
 - Şimdilik metotların sıra dışı durum fırlatmadığı varsayılacaktır

Örnek Bir Metot



Örnek Metotlar



```
int[] findPrimes(int limit){  
    int[] primes;  
    ...  
    return primes;  
}
```

```
int calculateArea(int width, int length){  
    return width * length;  
}
```

```
boolean isEven(int number){  
    boolean even = false;  
    if(number % 2 == 0)  
        even = true;  
    return even;  
}
```

Metotlara Erişim

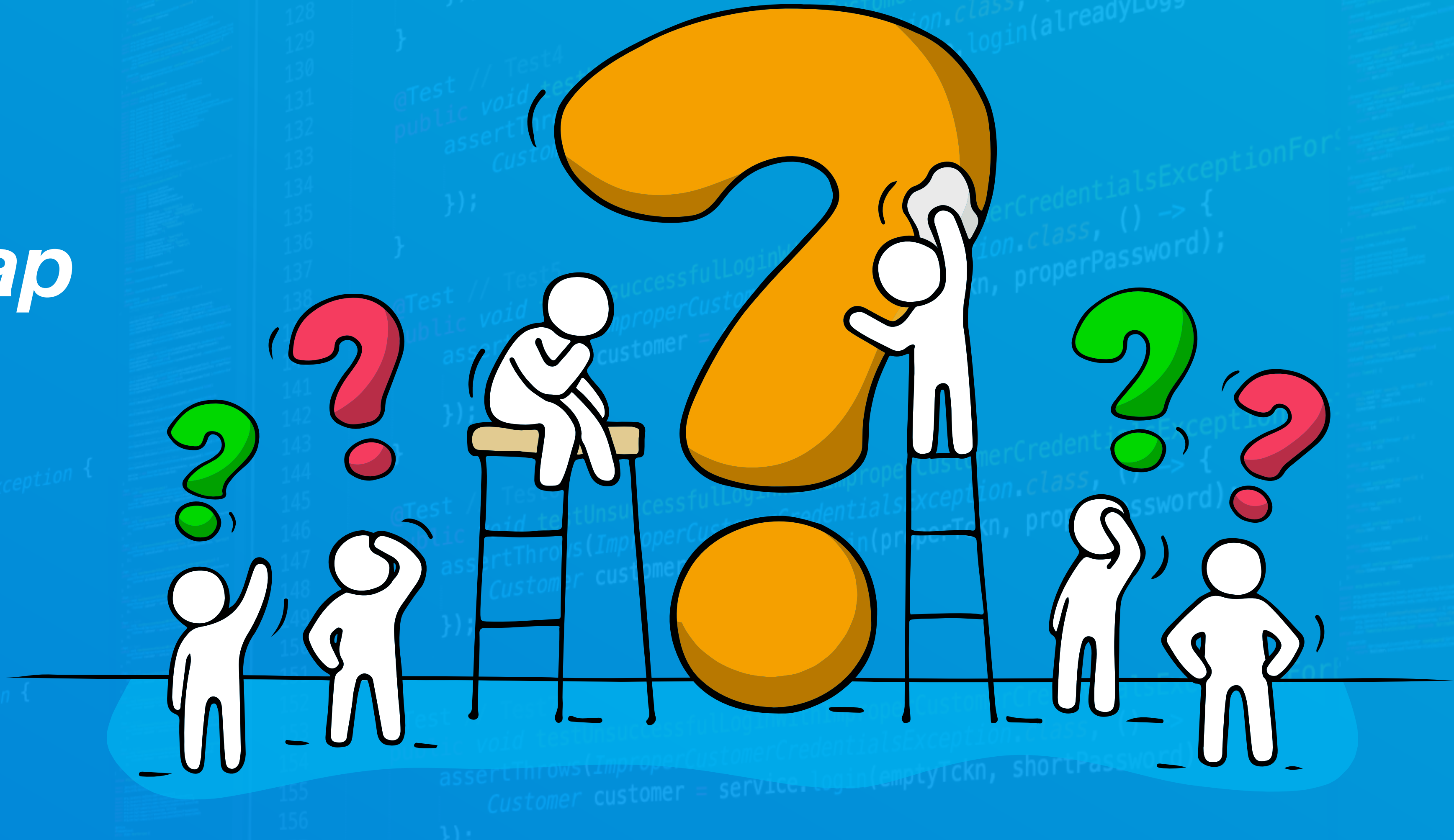


- Yaratılan nesnenin metotlarına, nesnenin referansı yoluyla erişilir
- Erişim “.” notasyonuyla olur:

`reference.method`

```
// No parameter passed  
reference.methodName();  
  
// With parameter(s) passed  
reference.methodName(...);
```

Soru ve Cevap Zamanı!





Parametreler

Parametreler - I



- Metot tanımındaki parametre listesi, virgülle ayrılmış, tip ve isim listesidir
- Parametreler, ilkel ya da referans tipten olabilir

```
returnType methodName(type1 param1, type2 param2, ...){ ...  
}
```

- Bir metot sıfır ya da daha fazla parametre alabilir
- Dolayısıyla metotlar parametre almak zorunda değildir

Parametreler - II



- Parametreler, metodun yerel değişkenleridir (local/stack variables), dolayısıyla çalışma zamanında metodun stackteki alanında bulunur

```
returnType methodName(type1 param1, type2 param2, ...){  
  
    // The scope of parameters  
  
}
```

- Parametrenin tipine göre değişkenin değeri stackte ya da heapte tutulur
- Parametrelerin kapsamı (scope), alt kapsamlar dahil tüm metottur

Formal ve Gerçek Parametreler - I



- Metot tanımında olan parametrelere **formal parametre (formal parameter)** denir
- Çalışma zamanında **metot çağrısı (method call)** yapılırken parametrelere karşılık **argüman geçmek (passing arguments)** gereklidir
- Metot çağrıldığında ona geçilen değerlere de **gerçek parametre (actual parameter)** denir
- Gerçek parametrelere **argüman (argument)** da denir

Formal ve Gerçek Parametreler - II



formal parameter



```
int square(int x){  
    int y = x*x;  
    return y;  
}
```

actual parameter



```
int i = 4;  
int iSquare = ref.square(i);  
long fiveSquare = ref.square(5);
```

actual parameter



Formal ve Gerçek Parametreler - II



- Metodun formal parametrelerinin sayısı ile geçen gerçek parametrelerin sayısı aynı olmalıdır
- Karşı gelen formal ve gerçek parametreler tip bakımından uyumlu olmalıdır
- Otomatik yükseltmeler çevirme (cast) operatörüne ihtiyaç duymadan yapılır
- Otomatik olarak dönüştürülmeyen gerçek parametreler için çevirme (cast) operatörü kullanılmalıdır, aksi halde derleyici hatası alınır

Formal ve Gerçek Parametreler - IV



- Referans parametresi için `null` da geçilebilir
 - Fakat referans beklenen yere `null` geçmek tehlikelidir:
 - `NullPointerException` tehlikesi!
- İlkel tipler için `null` geçilemez

Formal ve Gerçek Parametreler - V



formal parameter



```
int square(int x){  
    int y = x*x;  
    return y;  
}
```

actual parameter



```
int i = 4;  
int iSquare = ref.square(i);  
long fiveSquare = ref.square(5);  
ref.square(null) // Error!
```

actual parameter

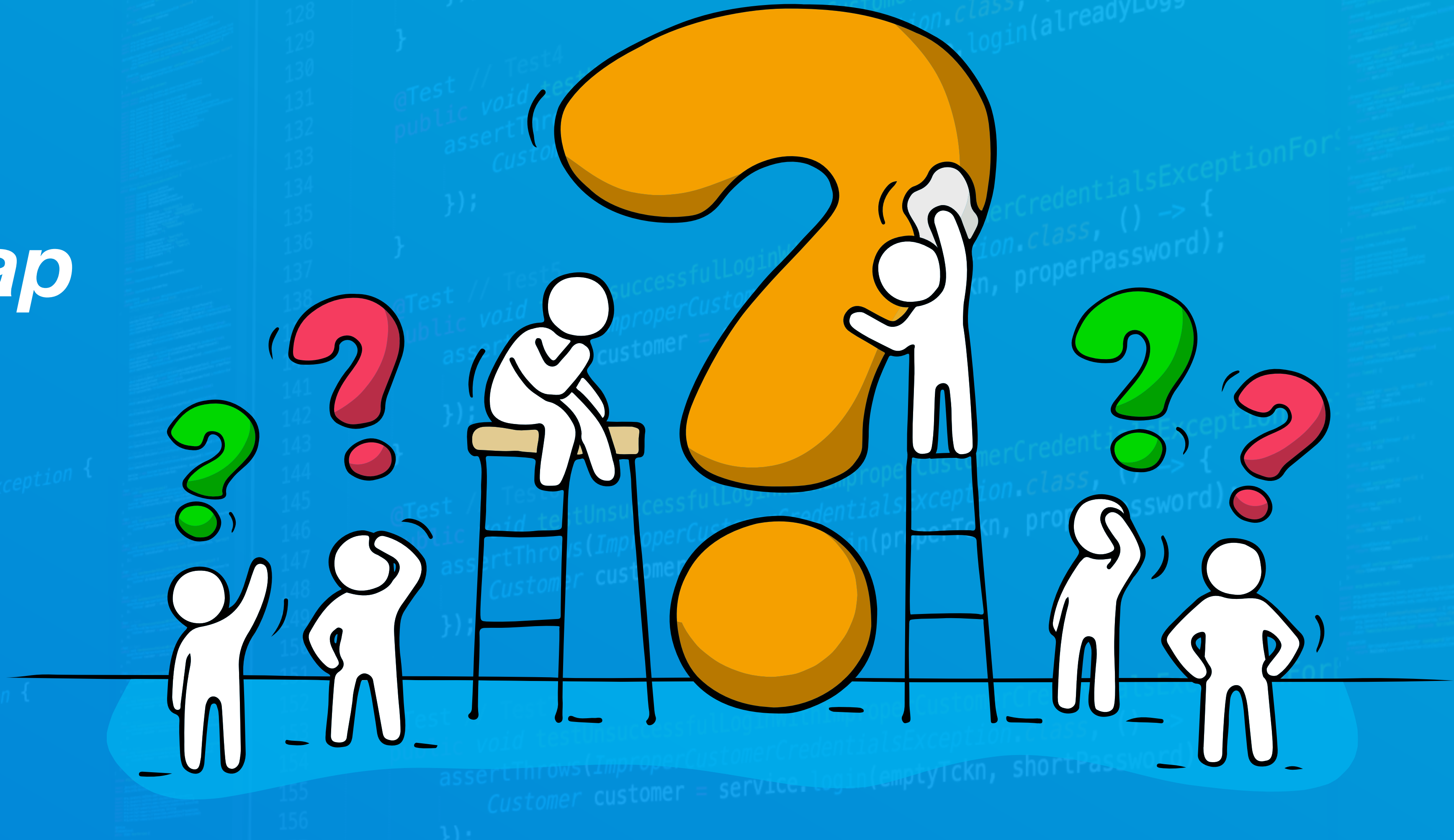


Varsayılan Değerli Parametreler



- Bazı programlama dillerinde metotların parametreleri, varsayılan (default) değere sahip olacak şekilde tanımlanabilir
- Bu durumda metot, parametre geçilmeden çağrılabilir ve metotta varsayılan değer kullanılır
- Java'da bu mümkün değildir, metotlar varsayılan değere sahip parametrelerle tanımlanamaz

Soru ve Cevap Zamanı!





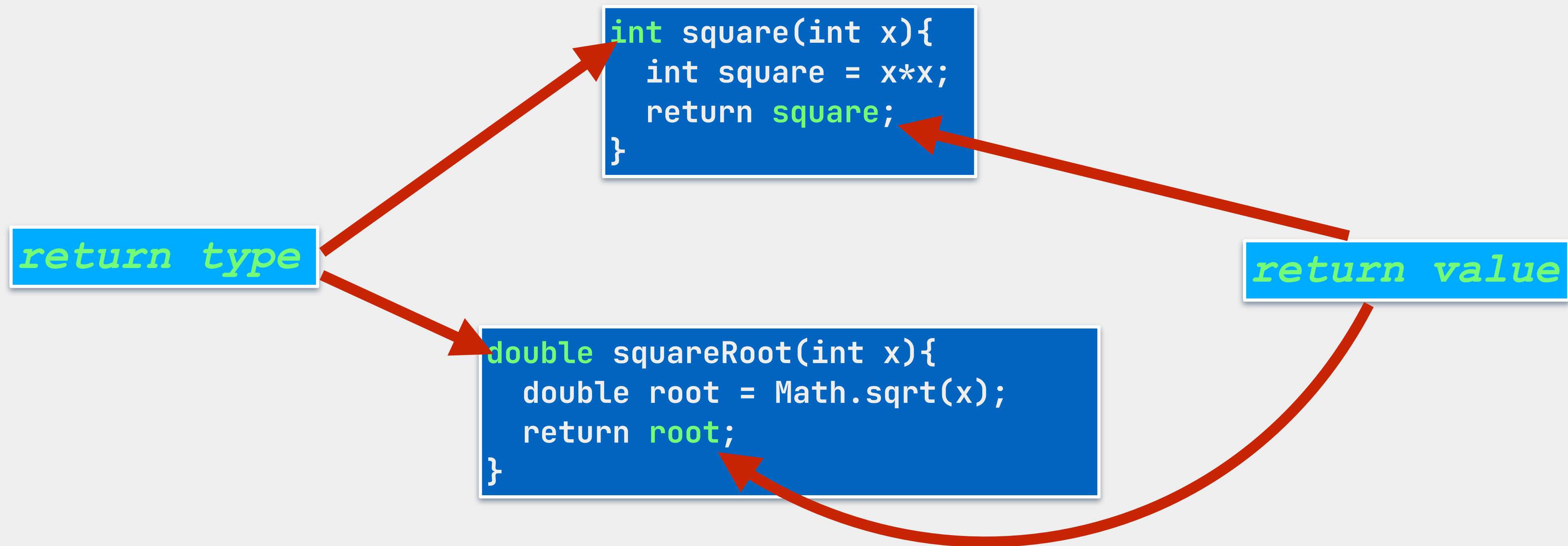
Dönüş Tipi

Dönüş Tipi - I



- Bir metot, çağrıldığı ortama değer döndürebilir
 - Bu durumda metot tanımında isimden önce dönüş tipi belirtilir
 - Dönüş tipi ile metot ismi arasına hiçbir şey giremez aksi takdirde derleme hatası olur
- Dönüş değerinin tipi ilkel olabileceği gibi referans da olabilir
- Metot, dönüş tipine uygun bir değer döndürmek zorundadır
- Bu amaçla **return** ve onu takip eden dönüş değeri kullanılır

Dönüş Tipi - II

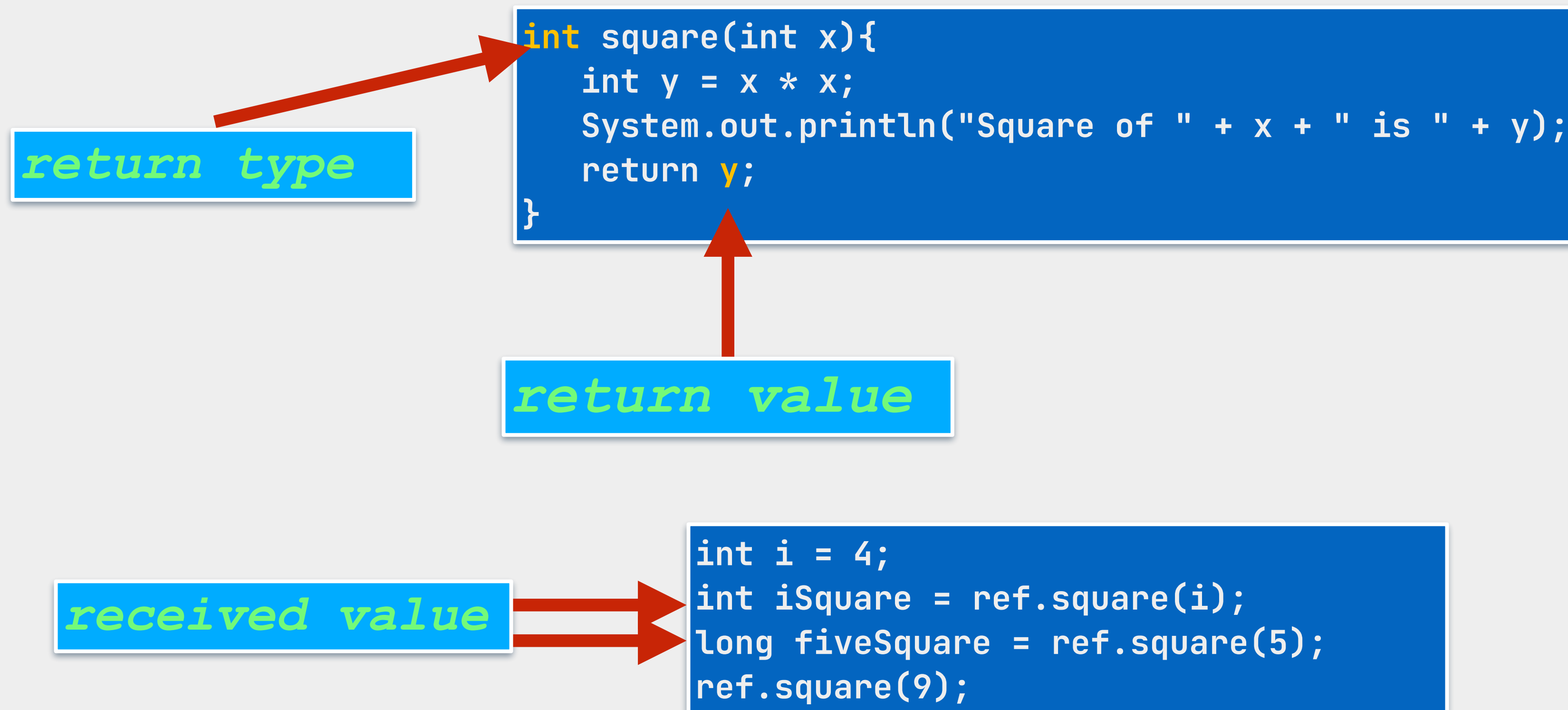


Dönüş Tipi - III



- Değer döndürülürken, döndürülen değer ile dönüş tipi arasında yükseltme ya da çevirme (cast) yapılabilir
- Bir metot en fazla bir değer döndürebilir
 - Go gibi dillerde fonksiyon ya da metotlardan birden fazla değer döndürülebilir
- Java'da ise birden fazla değer döndürmek söz konusuysa ya dizi gibi torba (collection) yapıları kullanılır ya da dönüş değerleri için nesne tasarlanır

Dönüş Tipi - IV

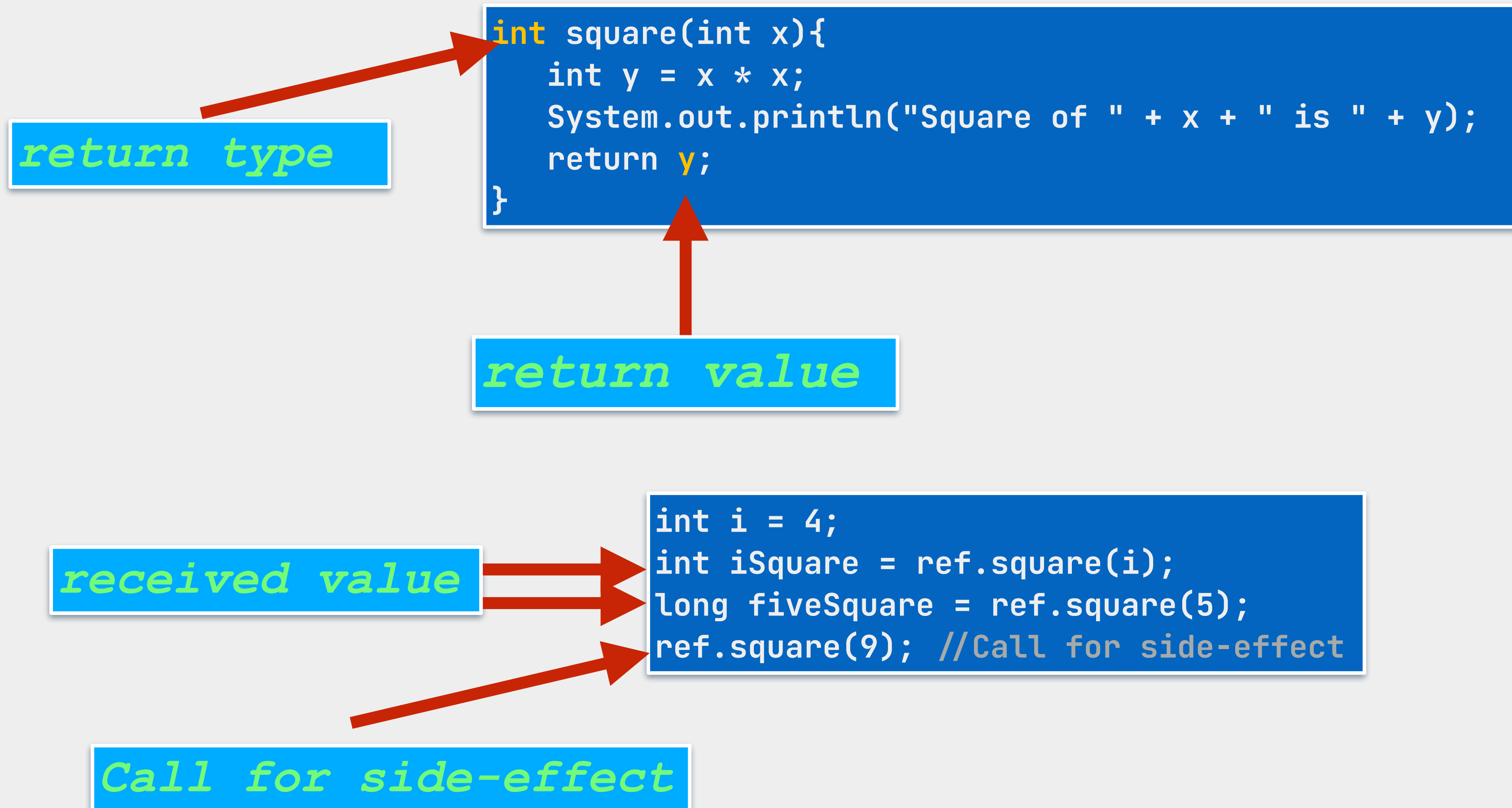


Dönüş Değerini Almak - I



- Metodun çağrıldığı ortam, metodun dönüş değerini almak istiyorsa, dönüş değerini, uygun tipteki bir değişkene atamalıdır
- Dönüş değeri ile ilgilenilmediği durumlarda atamaya da gerek yoktur
- Buna **bir metodu yan etkisi için çağırmak** (**calling a method for its side-effect**) denir

Dönüş Değerini Almak - II



Dönüş Değerini Almak - II



- Bir metot, bir değer döndürmek zorunda değildir
- Bu durumda dönüş tipi **void** olarak ifade edilir ve **return** hiç kullanılmayabilir ya da **return;** ile yetinilir
- **void** dönüş tipli metotlarda **return** kullanmamak iyi bir uygulamadır
- **void** döndüren metodun çağrıldığı ortamda, metot değer döndürmediğinden, dönüş değeri alınamaz, ilgili bir atama yapılamaz
- **void** döndüren metottan dönüş değeri almaya çalışmak bir hatadır

Dönüş Değerini Almak - IV



void return type

```
void print(int[] array){  
    for(int i : array)  
        System.out.println(i);  
    // return; // This is ok  
}
```

no return value

```
int[] array = new int[10];  
...  
ref.print(array);
```

no received value

```
int[] array = new int[10];  
...  
int i = ref.print(array); // Error!
```

Dönüş Tipi Olmalı mı? I



- Bir metodun dönüş tipi olmalı mı?
- Bu konu, metod tasarımının bir parçasıdır ve cevap için olası durumları göz önüne almak önemlidir
- Metodun anlamlı bir cevabı var mı?
- Eğer bir metod bir değer üretmiyorsa tabii olarak bir değer döndürmez
- Örneğin, `set`, `print` vb. metotlar bu tiptendir

```
void print(String s){...}  
void set(int x){...}
```

Dönüş Tipi Olmalı mı? II



- İstemcisinin değer beklediği, muhtemelen alıp başka yerde kullanacağı durumlarda, örneğin `get`, `getDate`, `getNextInt`, `readLine`, `create`, `find`, `search` vb. metotlar tabii olarak bir değer döndürürler

```
int get(){...}  
boolean isOpen(){...}  
boolean isEven(int i){...}  
String readLine(){...}  
int search(){...}  
Student create(){...}  
Object replace(Object o){...}  
double calculateTax(){...}
```

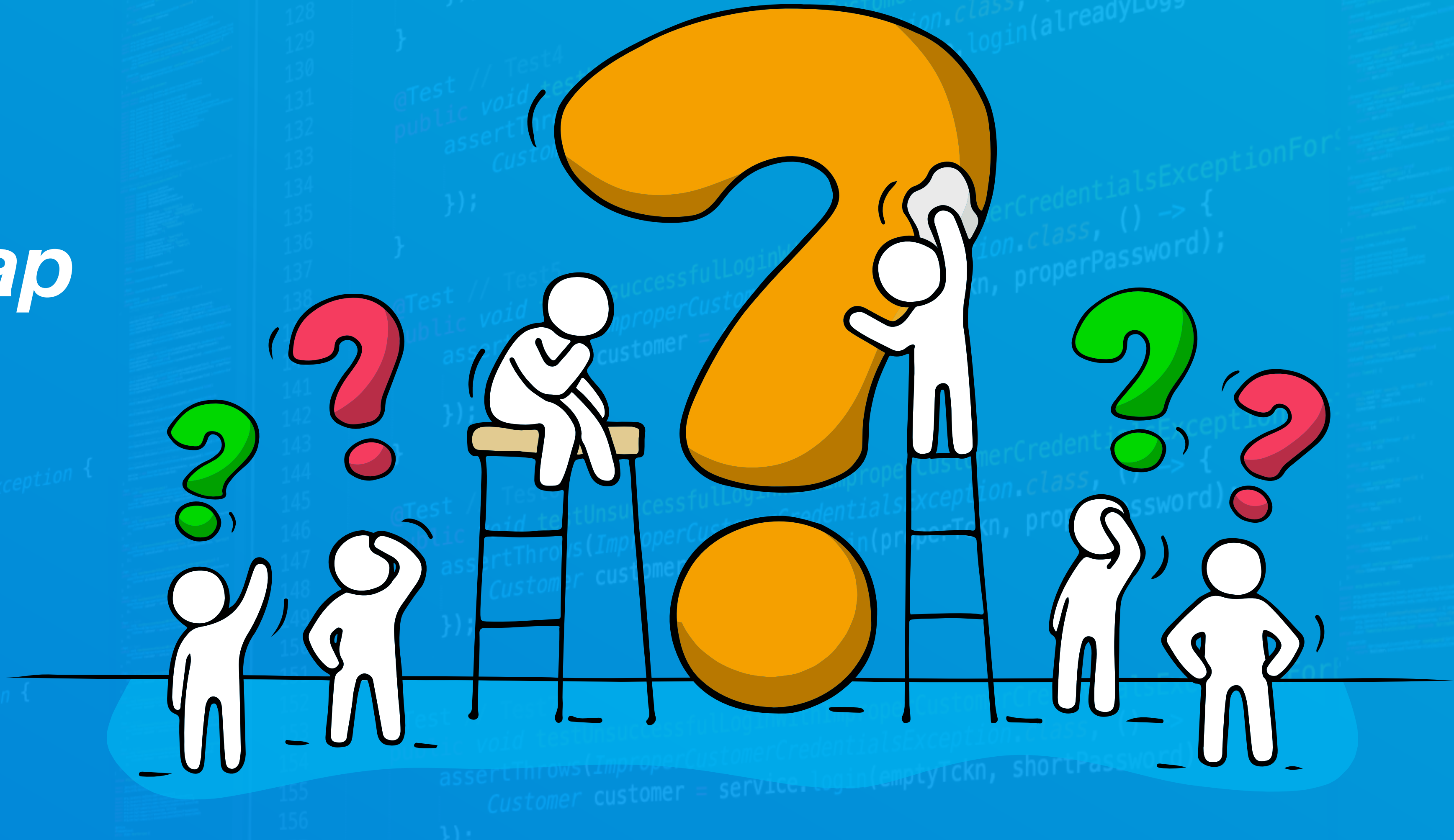
Dönüş Tipi Olmalı mı? III



- Bazen yapılan iş sonunda döndürülecek bir şey yoktur ama işin başarılı olup olmadığını belirtmek için **boolean** bir değer döndürülür
- Örneğin bir değeri dosya, veri tabanı vb. ortamlara kaydeden, torbalara ekleme yapan metotlar bu türdendir

```
boolean save(String line){...}  
boolean add(Object o){...}  
boolean start(){...}
```

Soru ve Cevap Zamanı!





Gerçekleştirme

Metodun Gerçekleştirilmesi



- Metodun gerçekleştirilmesi, işi yapan, sorumluluğu yerine getiren kısımdır
- Metodun **gerçekleştirilmesi (implementation)**, iki tane küme işareti **{ }** arasında, sıfır ya da daha fazla cümleden (statement) oluşur
- Metot gerçekleştirilmesine **metodun vücudu (method body)** ya da **kodu (code)** da denir

Metot Örnekleri - I



```
String selamSoyle(String kime) {  
    String cumle;  
    if (!kime.isEmpty())  
        cumle = "Selam " + kime + " :)";  
    else  
        cumle = "Selam " + world + " :)";  
    return cumle;  
}
```

```
boolean getFlag() { return true; } ⇒ boolean b = getFlag();  
  
float getNaturalLogBase() { ⇒ float e = getNaturalLogBase();  
    return 2.718f;  
}  
  
void doNothing() { return; } ⇒ doNothing();  
  
void doNothingElse() {} ⇒ doNothingElse();
```

Metot Örnekleri - II



```
// i is a numeric primitive  
int x = a.f(i);
```



```
// Possible methods in different classes  
byte f(int i){ ... }  
  
char f(int i){ ... }  
  
short f(int i){ ... }  
  
int f(int i){ ... }  
  
byte f(long l){ ... }  
...  
byte f(float f){ ... }  
...  
byte f(double d){ ... }  
...  
// 4 possible return types: byte, char, short and int  
// 7 possible parameter types: all primitives except boolean  
// Totally 4 * 7 = 28 possible method interfaces
```

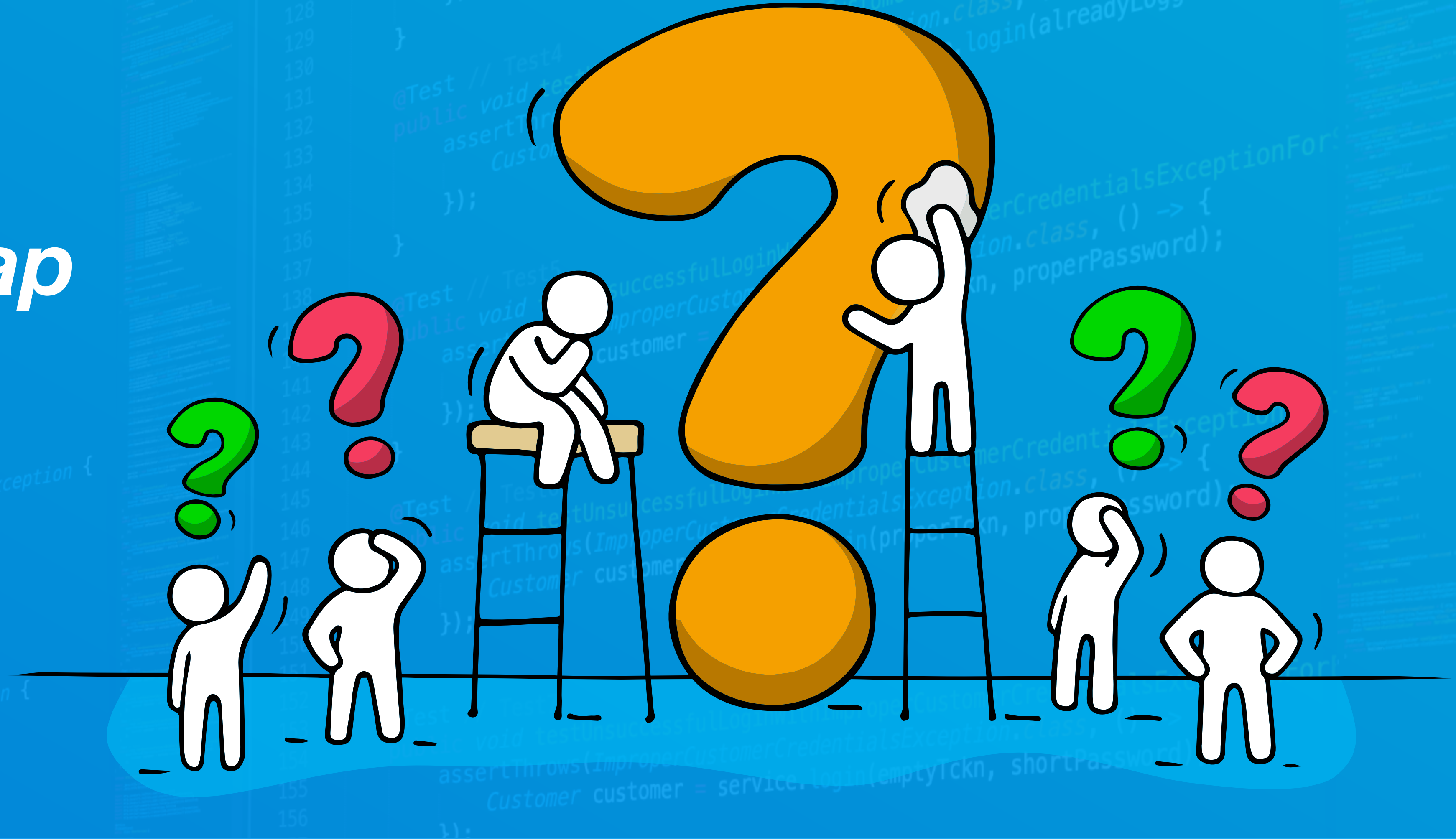
- Gerçekte daha fazla mümkün durum vardır, ilerledikçe anlayacaksınız

Calculator



- `calculator.Calculator`

Soru ve Cevap Zamanı!





Metotların Amaçları

Metotların Amaçları - I



- Metotların, nesnelerin şu üç tür davranışını yerine getirdiğini söylemiştik:
 - Nesnenin durumu hakkında bilgi vermek
 - Nesnenin durumunu değiştirmek
 - Hesaplama vb. türden bir iş yapmak
 - İş sonucunda sıklıkla bir çıktı ya da değer de üretilir

Metotların Amaçları - II



- Dolayısıyla kurucu metotlar dışında üç tür metot vardır:
- Nesnenin durumu hakkında bilgi veren (**accessor methods**), **getXxx** şeklindeki metotlar
- Nesnenin durumunu değiştiren (**mutator methods**), **setXxx** şeklindeki metotlar
- Bu iki tür metoda kısaca **get/set metotları** ya da **getter/setter** denir
- “Getter/setter”lar, nesnenin durumu ile ilgili al-ver yapan metotlardır

Metotların Amaçları - II



- Üçüncü tür metotlar, bir hesap yapma, bir süreci başlatma, bitirme ya da bir adımını ilerletme, bir kısıtı kontrol etme, bir dosyaya bir şey kaydetme, bir değer ya da nesne üretme vb. iş yapan metotlardır
- Örneğin, aşağıdaki metot isimleri bu türden metotların isimlerindendir:

`computePrice` `startCommunication` `authenticateUser`
`fetchAccountsByType` `createAccount` vs.

Metotların Amaçları - IV



- Metotlar, yukarıdaki üç şeyden bir ya da birkaçını yerine getirirler
- Nesne, özellikle de üçüncü türden metotlarda sorumluluklarını yerine getirirken başka nesnelerden hizmet alabilir (delegation)
- Bu şekilde büyük sorumluluklar, birden fazla nesnenin el birliğiyle yerine getirdiği işler olur
- Her nesne kendi üzerine düşen küçük bir parçayı yerine getirir



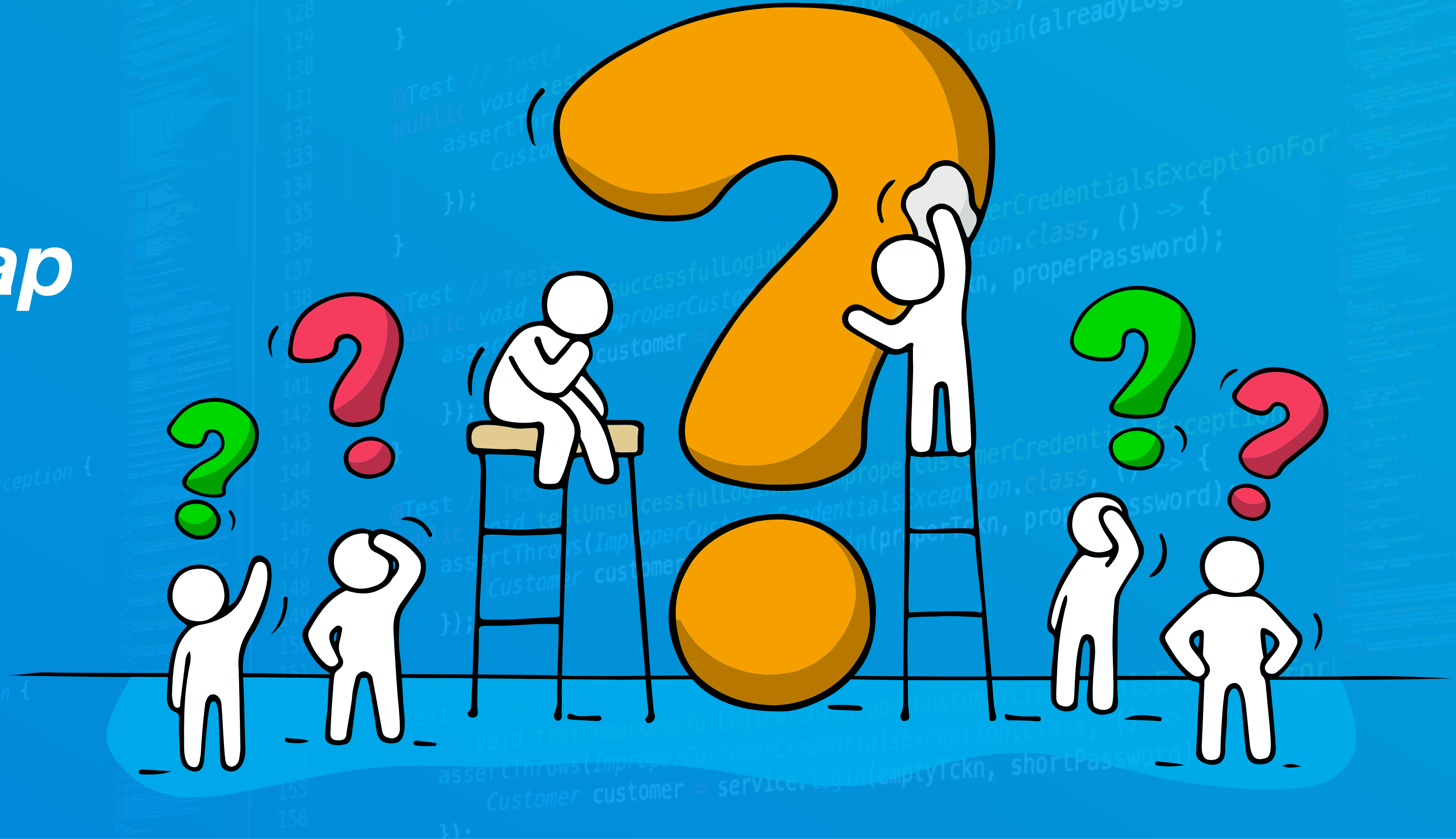
- Yazılım tasarımı, temelde büyük işleri küçük parçalara bölmek ve onları nesneler arasında paylaşmaktır

FootballPlayer



- `player.FootballPlayer`

Soru ve Cevap Zamanı!





Ödev

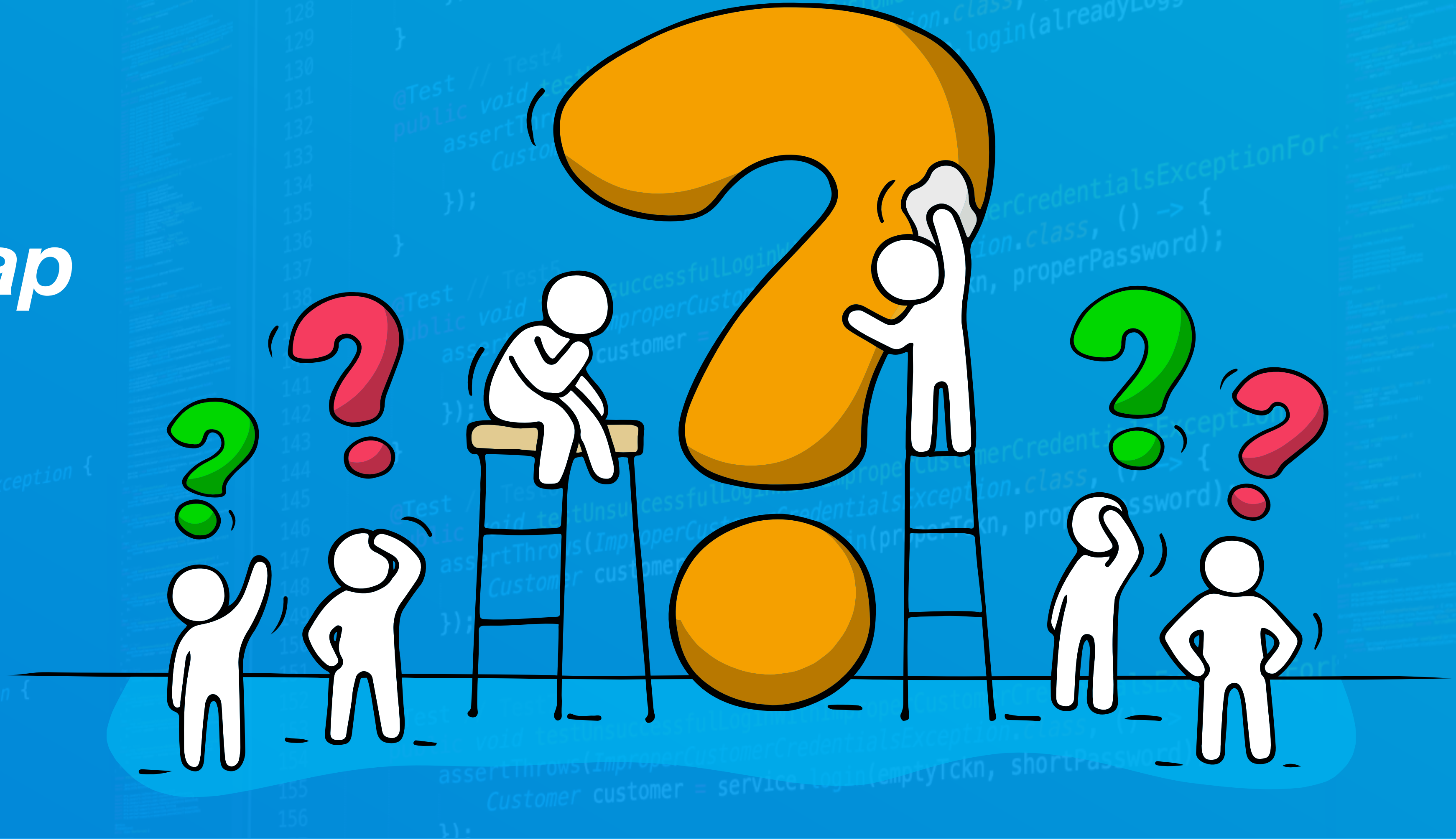


- `calculator.Calculator` örneğindeki `Calculator`'a aşağıdaki işleri yapan yeni metotlar yazın ve `main` metottan çağırarak test edin
 - a. Mod alan (kalanı bulan)
 - b. Üs alan (`base ^ power`)
 - c. Faktöriyel alan



- `Calculator` sınıfını kullanarak, kullanıcısından aldığı operatör, ilk argüman ve ikinci argüman gibi girdilerle hesap yapıp cevabını bildiren bir `CalculatorApp` yazın

Soru ve Cevap Zamanı!





get/set metotları

Get ve Set Metotları - I



- **get/set metotları (getters & setters)** nesnenin durumuna ulaşmak ya da değiştirmek amacıyla kullanılan standart nesne metotlarıdır
- Amaçları:
 - **get** metodu nesnenin durumunun parçası olan bir nesne değişkeninin (instance variable) değerini geri döndürmek
 - **set** metodu nesnenin durumunun parçası olan bir nesne değişkenine (instance variable) değer atamak

Get ve Set Metotları - II



- **get/set** metotları tabii olarak **Lower Camel Case**'e uygun olarak yazılır
- Eğer nesne değişkeni `int field` ise **get/set** metotları **Lower Camel Case**'e uygun olarak şöyle olur:
 - `int getField()`: argüman almaz ama `int` dönüş tipine sahiptir
 - `void setField(int value)`: `int` bir argüman alır ve `void` döndürür
 - `boolean` alanların `get` metodu `isField` olarak da yazılabilir

Get ve Set Metotları - II



- IDE'ler `get/set` metotlarını otomatik olarak oluşturan kısa yollara sahiptir

```
class IntSetterGetter{
    int value; //A field

    //setter
    void setValue(int newValue){
        value = newValue;
    }

    //getter
    int getValue(){
        return value;
    }
}
```

```
class BooleanSetterGetter{
    boolean open; //A field

    //setter
    void setOpen(boolean state){
        open = state;
    }

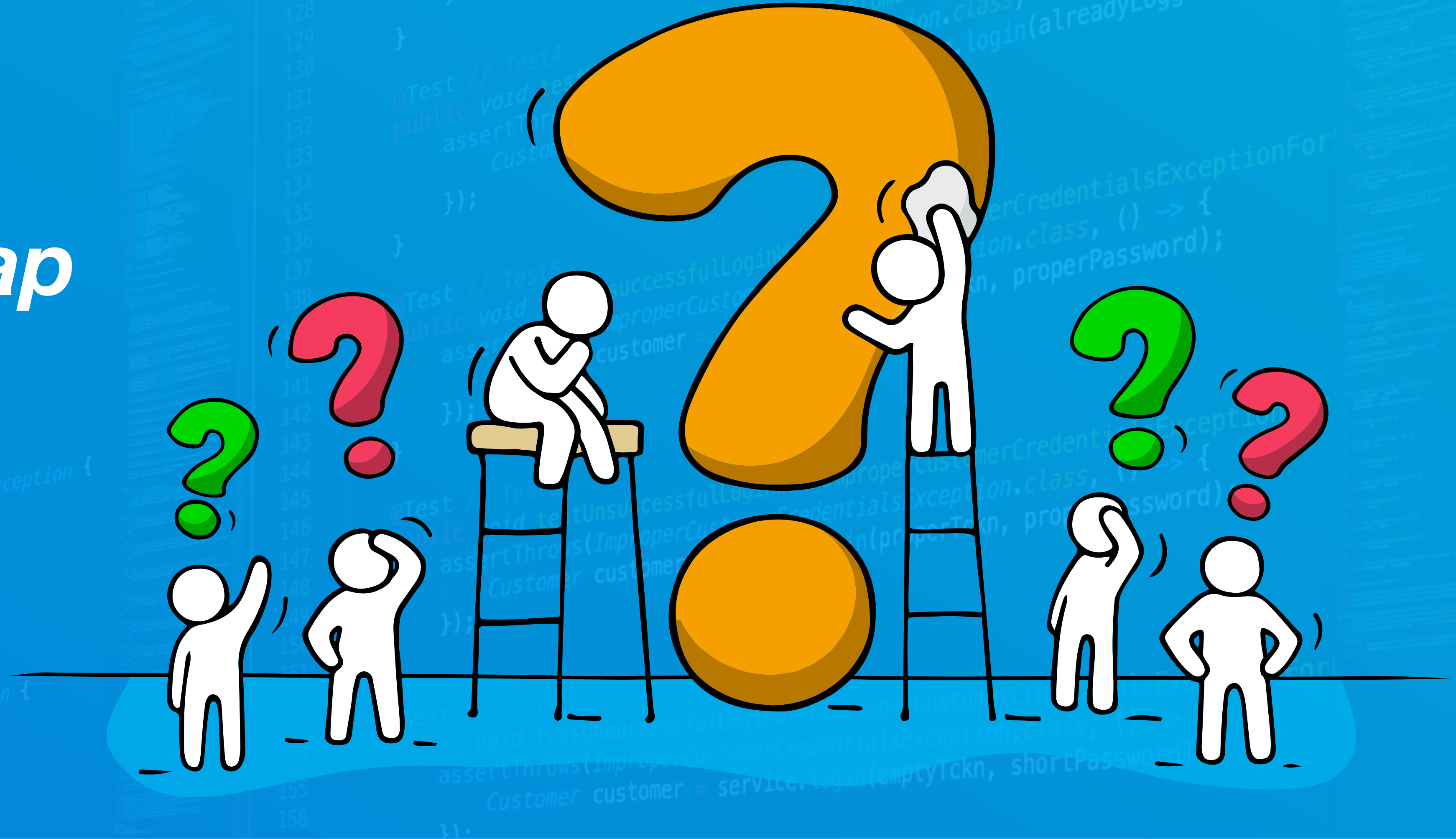
    //getter
    boolean isOpen(){
        return open;
    }
}
```

Window



- Window

Soru ve Cevap Zamanı!





İmza ve Arayüz

İmza ve Arayüz - I



- Bir metodun, isim ve parametre listesinden oluşan bilgisine imza (signature) denir
- Dönüş değeri tipi ve fırlattığı sıra dışı durumlar imzaya dahil değildir
- Bir metodun, isim, parametre listesi, dönüş değeri tipi ve fırlattığı sıra dışı durumlardan oluşan bilgisine arayüz (interface) denir

```
<modifier>* <return type> <name>(<Parameter>*) throws <exception>* //signature  
<niteleyici>* <dönüş tipi> <isim>(<Parametre>*) throws <exception>* //imza  
  
<modifier>* <return type> <name>(<Parameter>*) throws <exception>* //interface  
<niteleyici>* <dönüş tipi> <isim>(<Parametre>*) throws <exception>* //arayüz
```

İmza ve Arayüz - II



```
public String selamSoyle(String kime) //signature  
public String selamSoyle(String kime) //interface
```

```
public double squareRoot(double arg) throws IllegalArgumentException //signature  
public double squareRoot(double arg) throws IllegalArgumentException //interface
```

- Bu örneklerde bulunan **public** ya da **throws** gibi anahtar kelimeleri ileride ele alacağız.

Arayüz (Interface)



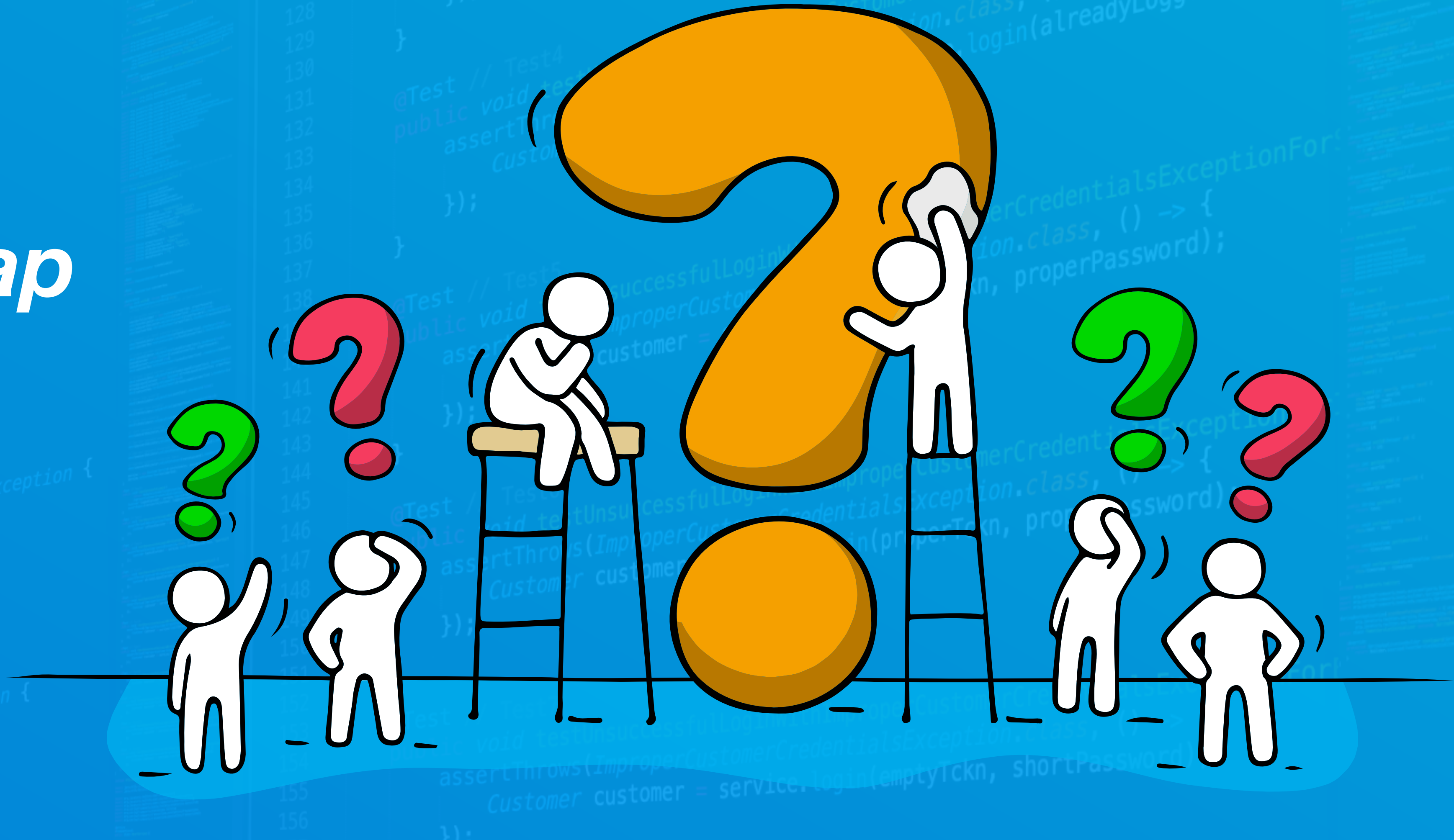
- Bir metot ancak arayüz (interface) bilgisiyle çağrılabilir
- Arayüz bilgisini daha anlaşılır kılmak amacıyla **JavaDoc** ile arayüz dokümantasyonu (interface documentation) yapılabilir
- Bir sınıfta imzası, dolayısıyla da arayüzü aynı olan iki tane metot olamaz, bu bir derleyici hatasıdır

Nesnenin Arayüzü



- Bir nesnenin sahip olduğu metot arayüzlerinin tamamına, **nesnenin arayüzü (the interface of the object)** denir
- Dolayısıyla nesne arayüzü, nesnenin sınıfında tanımlanan metot arayüzlerinin toplamıdır
- Nesnenin arayüzü, o nesnenin verdiği hizmetlerin ya da alabileceği mesajların bütünüdür

Soru ve Cevap Zamanı!



Metotların Özellikleri



Metotların Özellikleri



- Şimdi metotların bazı özelliklerini ele alacağız:
 - Metotlarda çoklu kullanım
 - Değişken sayıda argümanlı metotlar
 - Parametre geçme
 - **final** parametre



Metotlarda Çoklu Kullanım

Overloading - I



- Daha önce + işlemcisinin (operator), sayı, **String** ve karakter işlenenlerle (operand) kullanıldığında farklı davrandığını görmüştük
- Bu durum + işlemcisinin **çoklu kullanımı (overloading)** olarak isimlendirilmişti
- Metotlarda çoklu kullanım, isimleriyle olur, yani bir sınıfta birden fazla metod aynı isme sahip olabilir
- Bu tür metotlara **çoklu kullanımlı metotlar (overloaded methods)** denir

Overloading - II



- Overloading, aynı işi farklı parametrelerle yapan metotları isimlendirmek için kullanılır; çünkü bu kolaylık sağlar
- Overload edilen metotların isimleri aynı olduğundan, parametre listesi, sayı ve/veya tip bakımından farklı olmalıdır
- Çünkü bir sınıfta imzası aynı olan iki tane metot olamaz

```
class Adder{  
    int add(byte first, byte second) {...}  
    int add(short first, short second) {...}  
    int add(char first, char second) {...}  
    int add(int first, int second) {...}  
    long add(long first, long second) {...}  
    float add(float first, float second) {...}  
    double add(double first, double second) {...}  
}
```

Overloading - III



- Java kütüphanelerinde de pek çok overloaded metot vardır; örneğin:
 - `System` sınıfındaki `out` referansı `PrintStream` tipindedir ve `print()` metodunun 9, `println()` metodunun 10 overloaded haline sahiptir
 - Çünkü `print()` 'in, geçilen 6 ayrı ilkel tip ve 3 referans tipi konsola yazarken, `println()` 'in bunlara ek bir de hiç argüman almayan overloaded hali vardır
 - <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/io/PrintStream.html>

Adder



- `overloading.Adder`

Overloading ve Overriding



- **Overloading** ve **overriding** farklı kavramlar ve mekanizmalardır, karıştırılmamalıdır
- Overriding ileride ele alınacaktır

Bir Soru - I



- **Soru:** İmzası aynı olup da dönüş tipleri farklı olan metotlarla overload yapılamaz. Neden?

```
// Error!  
boolean method(int i, long l){  
    return true;  
}  
  
int method(int x, long y){  
    return 5;  
}
```

Bir Soru - II



- Çünkü bir metod, döndürdüğü değer alınmadan da çağrılabilir
- Yan etkisi için çağırma (side-effect call)

```
// Error!  
boolean method(int i, long l) {  
    return true;  
}  
  
int method(int x, long y) {  
    return 5;  
}  
...  
method(5, 8L); // Which one is called?
```

OverloadingOverReturnType



- `overloading.OverloadingOverReturnType`

İlkel Tiplerle Çoklu Kullanım



- Metotlara parametre geçilirken gerektiğinde otomatik yükseltme (promotion) yapılır
- Birden fazla alternatif üst tip varsa ne olur?
- En az yükseltme gerektiren tipe mi yoksa standart tipe mi?
- Hiçbir uygun üst tip yoksa?

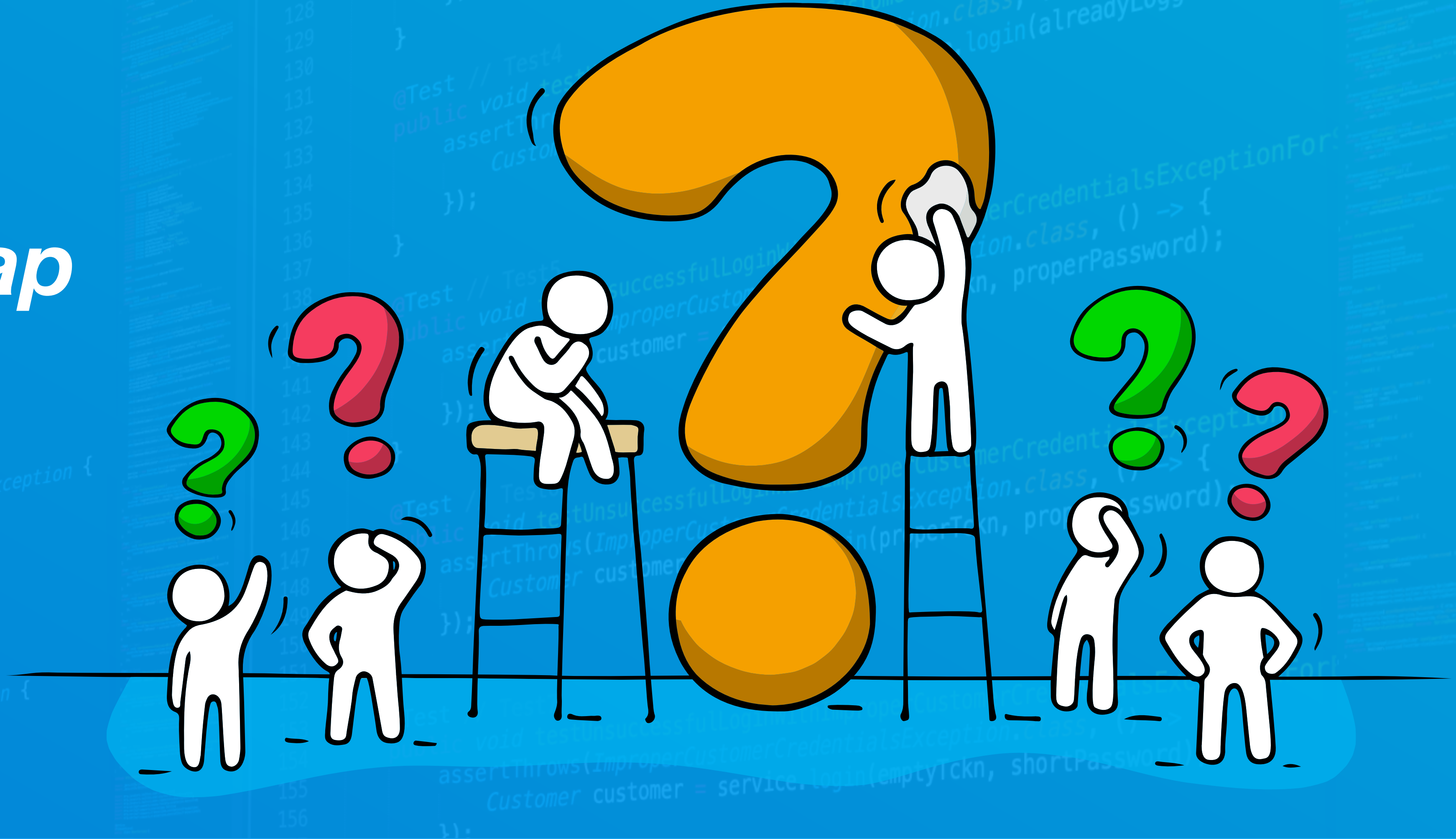
```
byte b = 3;  
x.method(b); // Which one is called?  
  
void method(short i) {...}  
void method(int i) {...}  
void method(long i) {...}
```

PrimitiveOverloading1 & PrimitiveOverloading2



- `overloading.PrimitiveOverloading1`
- `overloading.PrimitiveOverloading2`
- Bu örnekte derleyicinin hangi durumda neden hata verdiğine dikkat edin

Soru ve Cevap Zamanı!





Değişen Sayıda Argümanlı Metotlar

Değişen Sayıda Argümanlı Metotlar - I



- Bir metodun aldığı argüman sayısına **arity** denir
- En başta Java'da bir metodun alabileceği argüman sayısı, metot tanımlanırken belirlenirdi ve daima sabit kalırdı (fixed arity)
- Fakat Java 5 ile **değişen sayıda argümanlı metot (variable arity method)** özelliği geldi
- Bu tür metotlara, aldıkları argüman sayısı değiştiğinden dolayı **değişen argümanlı metotlar (var-args methods)** da denir

Değişen Argümanlı Metotlar - II



- Bir sınıfta bir metodun aynı tipten farklı sayıda parametre alan çoklu kullanılmış (overloaded) halleri olabilir
- Fakat farklı sayılarda parametre alan overloaded metotlar yazmak zahmetli ve bitmeyen bir iştir
- Çünkü kaç tane argüman geçilebileceğini bilemezsiniz

```
double calculateAverage(int i, int j) {}  
double calculateAverage(int i, int j, int k){}  
double calculateAverage(int i, int j, int k, int l){}  
double calculateAverage(int i, int j, int k, int l, int m) {}  
...  
// Bu şekilde sonsuz tane metot yazılabilir!
```

Değişen Sayıda Argümanlı Metotlar - II



- Çözüm olarak çok sayıda metot yerine dizi alan tek bir metot yazılabilir:

```
double calculateAverage(int[] numbers){ }  
...  
int[] a1 = {1, 2};  
calculateAverage(a1);  
  
int[] a2 = {1, 2, 3, 4, 5, 6, 7, 8, 9};  
calculateAverage(a2);  
  
int[] a3 = new int[28];  
... // Initialization  
calculateAverage(a3);
```

- Bu durumda metodu çağırarak için daima dizi oluşturmak gerekir

Değişen Argümanlı Metotlar - IV



- Değişen argümanlı metotlar, hem metodu yazana, hem de metodu çağırana kolaylık için tasarlanmıştır
- Java 5 sürümüyle gelen **değişen argümanlı metotların (var-args methods)** parametresi **...** ile tanımlanır
- Bu şekilde tanımlanan metot, farklı sayıda argüman geçilerek çağrılabilir

```
double calculateAverage(int ... numbers) {  
    double sum = 0;  
    for(int i : numbers)  
        sum += i;  
    double average = sum/numbers.length;  
    return average;  
}
```

```
calculateAverage(1,2,3);  
calculateAverage(1,4,7,9,11,14,21,29,35);  
calculateAverage(); // numbers.length = 0 → NaN!
```

Değişen Argümanlı Metotlar - V



- Değişen argümanlı metotlarda `...` ile geçilen parametre, metotta dizi olarak görünür
- Değişen argümanlı metot birden fazla argüman alıyorsa dizi argümanı en son geçilen argüman olmalıdır
- Bu yüzden bir metoda sadece bir tane `...` argümanı geçilebilir
- Eğer bir metot `...` parametresi ile tanımlanıyorsa `[]` ile tanımlanan başka bir diziyi parametre olarak alamaz

Değişen Argümanlı Metotlar - VI



- Değişen argümanlı metotlar, hiç parametre geçmeden de çağrılabilir
- Bu durum, 0 uzunluğunda bir dizi geçmeye eşittir

```
double calculateAverage(int ... numbers) {  
    double sum = 0;  
    for(int i : numbers)  
        sum += i;  
    double average = sum/numbers.length;  
    return average;  
}
```

`calculateAverage();`

```
double calculateAverage(int ... numbers) {  
    if(numbers.length == 0) return 0;  
    double sum = 0;  
    for(int i : numbers)  
        sum += i;  
    double average = sum/numbers.length;  
    return average;  
}
```

- Ama bu metot ile 0 uzunluğunda bir dizi geçildiğinde, 0.0/0 işleminden dolayı **NaN** değerini geri döner

Değişen Argümanlı Metotlar - VII



- ... de **[]** gibi tip ile isim arasında herhangi bir yerde olabilir
- Örneğin, **main** metot aşağıdaki şekillerde yazılabilir

```
public static void main(String[] args)
public static void main(String [] args)
public static void main(String []args)
public static void main(String args[])

public static void main(String... args)
public static void main(String ... args)
public static void main(String ...args)
public static void main(String...args)
```

```
void main(String[] args)
void main(String [] args)
void main(String []args)
void main(String args[])

void main(String... args)
void main(String ... args)
void main(String ...args)
void main(String...args)
```

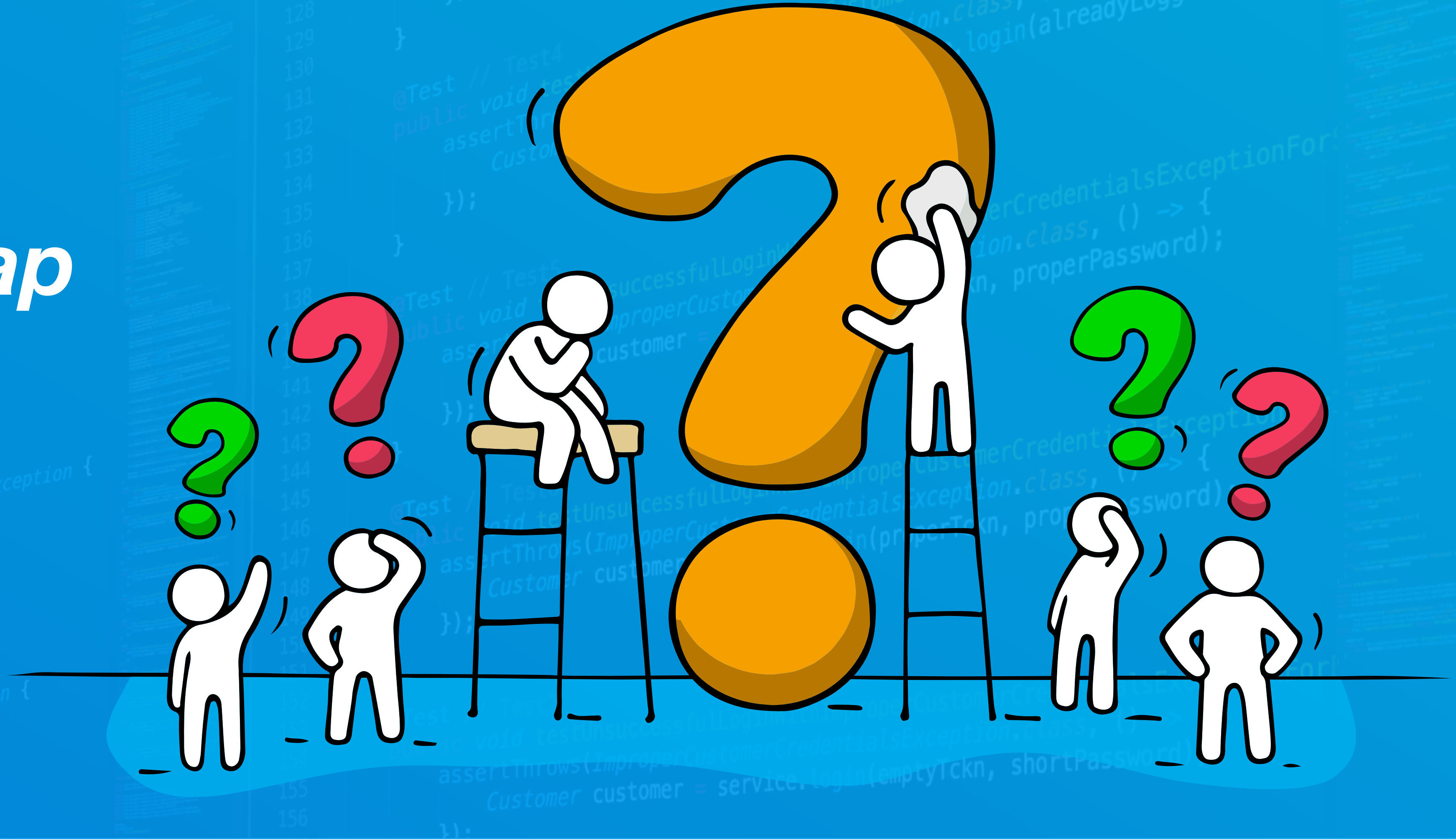
- Dikkat, **[]** isimden sonra gelebilirken ... isimden sonra gelemmez!

VarArgMethod & VarArgMethodProblem



- `varArg.VarArgMethod`
- `varArg.VarArgMethodProblem`

Soru ve Cevap Zamanı!





Parametre Geçme

Parametre Geçme - I



- Biliyoruz ki metot çağrılırken, metoda gerçek parametreler ya da argümanlar geçer ve bu argümanlar metot içinde formal parametrelerle temsil edilir
- Geçilen argümanlarla formal parametreler arasındaki ilişki nasıldır?
 - İlkel tipler ve referanslar nasıl geçer?
 - Örneğin hep nesne geçmekten bahsedilir ama aslen nesneler değil, referansları geçer

Parametre Geçme - II



- Örneğin yandaki kod parçasında `i`'nin değeri ne olur?
- 6 mı olur yoksa 5 olarak mı kalır?

```
void f(int j){  
    j++;  
}  
  
void main(){  
    int i = 5;  
    f(i);  
    System.out.println(i); // i?  
}
```

Parametre Geçme - III



- Benzer başka bir kodda `a.i`'nin değeri ne olur?
- 6 mı olur yoksa 5 olarak mı kalır?

```
void f(A aa){  
    aa.i++;  
}  
  
void main(){  
    A a = new A();  
    f(a);  
    System.out.println(a.i); // a.i?  
}  
  
class A{  
    int i = 5;  
}
```

Değerle Geçmek (Pass-by-Value) - I



- **Java'da parametreler, değerleri ile geçilir (pass-by-value)**
 - Bir metoda bir ilkel tipte değişken geçilirken, parametrenin değeri, gerçek değişkenin değeri olarak belirlenir
 - Benzer şekilde metoda geçilen bir referans ise, bu durumda o referansın değeri, parametre değeri olarak belirlenir; referansın değeri ise zaten nesnenin adresidir
- Dolayısıyla, her halükarda metoda geçilen şey, gerçek argümanın değeridir

Değerle Geçmek (Pass-by-Value) - II



- Bir metoda basit tipli değişken geçilirken, parametrenin değeri gerçek değişkenin değeri olur
- Dolayısıyla örnekte *j*'nin değeri artar ama *i*'ninki değişmez, 5 olarak kalır

```
... void f(int j){// j = i assignment!  
    j++;  
}  
  
... void main(String[] args){  
    int i = 5;  
    f(i);  
    System.out.println(i); // i?  
}
```

A red curved arrow originates from the `f(i)` call in the `main` method and points to the `j` parameter in the `f` method, illustrating that a copy of the value of `i` is passed to `j`.

Değerle Geçmek (Pass-by-Value) - II



- Bir metoda referans geçilirken, parametrenin değeri, gerçek değişkenin değeri olarak belirlenir
- Böylece parametre, referansın gösterdiği nesneyi gösterir hale gelir, yani iki referans da aynı nesneyi gösterir
- Örnekte `aa.i`'nin de `a.i`'nin de değeri artar çünkü ikisi de aynı nesnenin alanıdır

```
... void f(A aa){ // aa = a assignment!  
    aa.i++;  
}  
  
... void main(String[] args){  
    A a = new A();  
    f(a);  
    System.out.println(a.i); // a.i?  
}  
  
class A{  
    int i = 5;  
}
```

ParameterPassing



- `passing.ParameterPassing`

Değerle Geçmek (Pass-by-Value) - IV



- Java'da argümanlar değerleriyle geçildiklerinden, bir metoda geçilen bir ilkel değişkenin değeri metotta değişse bile, asıl değerinde bir değişiklik olmaz
- Çünkü değişen parametredir, gerçek değişkenin değeri değişmez
- Gerçek değişkenin değerini değiştirmek için değişen değeri metottan geri döndürmek ve geçilen değişkene atamak gereklidir

```
... int f(int j){  
    j++;  
    return j;  
}  
  
... void main(String[] args){  
    int i = 5;  
    i = f(i);  
    System.out.println(i); // 6  
}
```

Değerle Geçmek (Pass-by-Value) - V



- Nesnelerde ise, metottaki parametre ile yeni bir referans oluşturulur ve her iki referans da aynı nesneyi gösterir ve bu şekilde birbirlerinin değişikliklerinden haberdar olurlar
- Dolayısıyla metotta değeri değiştirilen ilkel tipten bir değişken bile olsa çağrılan yerde bu değişiklik görülür

```
... void f(A aa){ // aa = a assignment!  
    aa.i++;  
}  
  
... void main(String[] args){  
    A a = new A();  
    f(a);  
    System.out.println(a.i); // 6  
}  
  
class A{  
    int i = 5;  
}
```

Referans ile Geçmek (Pass-by-Reference)



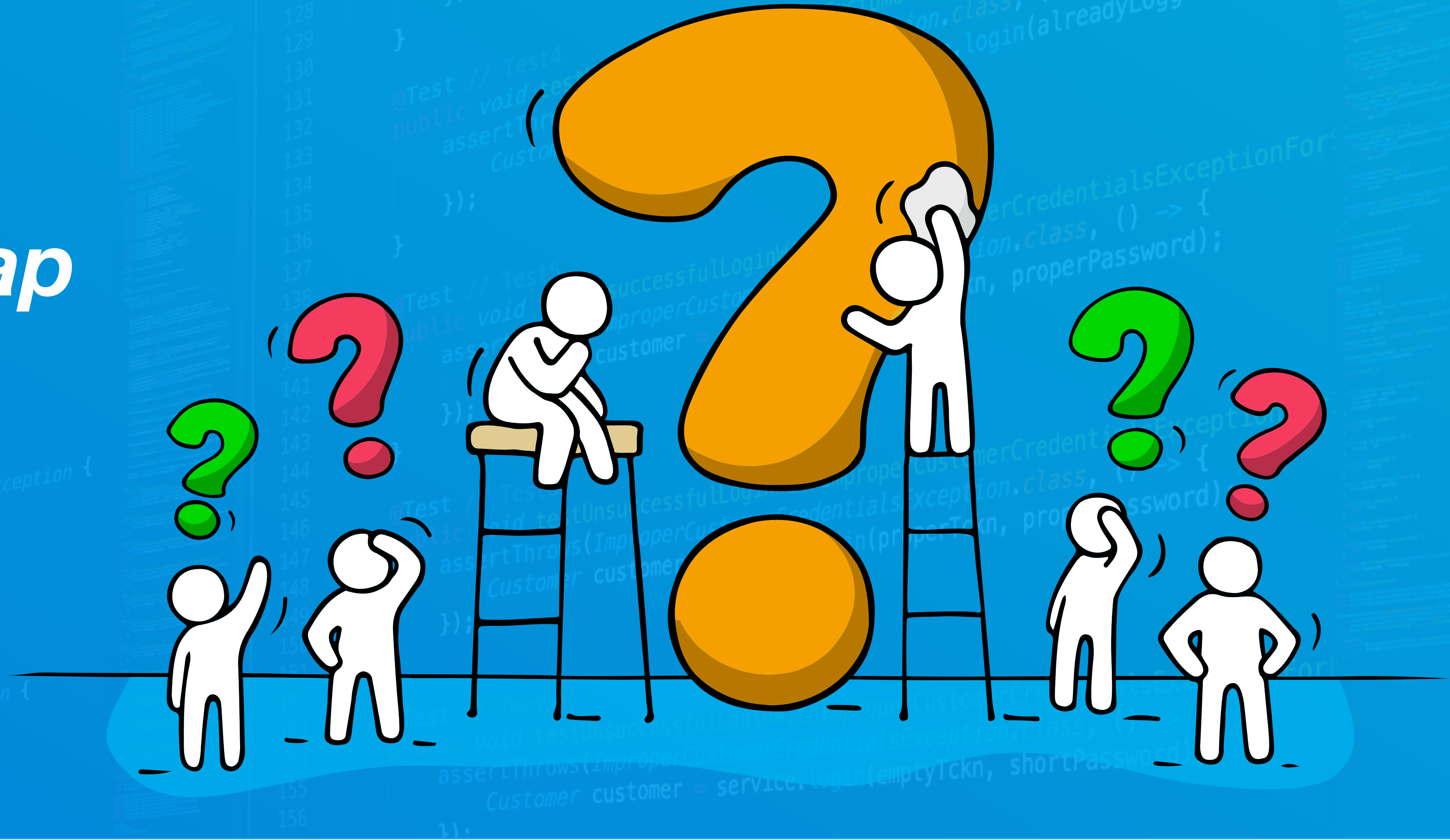
- Bruce Eckel (**Thinking in Java**'nın yazarı) gibi bazı yazarlar, anlamayı kolaylaştırmak için “Java’da nesneler referanslarıyla geçilir” derler
- Çünkü referansların değeri yani adresi geçilmektedir
- Ama literatürde **referans ile geçmek (pass-by-reference)** farklı bir mekanizmayı ifade eder

ObjectPassing



- `passing.ObjectPassing`

Soru ve Cevap Zamanı!





final
Parametre

final Değişkenler



- **final** anahtar kelimesi ile daha önce değişkenlerin nasıl sabitelere (constant) dönüştürüldüğünü görmüştük
- **final** ile tanıtilan değişkenlerin değeri değişemez

```
final int i = 5;  
i = 8; // Compile-time error.  
final String s = "Java";  
s = "I love Java"; // Compile-time error.
```

```
final int i; // Blank final is ok.  
i = 5;  
i = 8; // Compile-time error.  
final String s; // Blank final is ok.  
s = "Java";  
s = "I love Java"; // Compile-time error.
```

final Parametreler - I



- **final** ile bir metoda geçilen bir parametrenin, sadece o metotta sabite olması sağlanabilir
- Bu amaçla parametre metotta tanıtılırken **final** kullanılır
- **final** tanımlanan bir parametrenin değeri metotta değişemez

```
void speedUp(final int newSpeed){  
    newSpeed = 75;    // Compile-time error.  
    speed = newSpeed;  
}  
  
void setOwner(final String newOwner){  
    newOwner = "akin"; // Compile-time error.  
}
```

final Parametreler - II



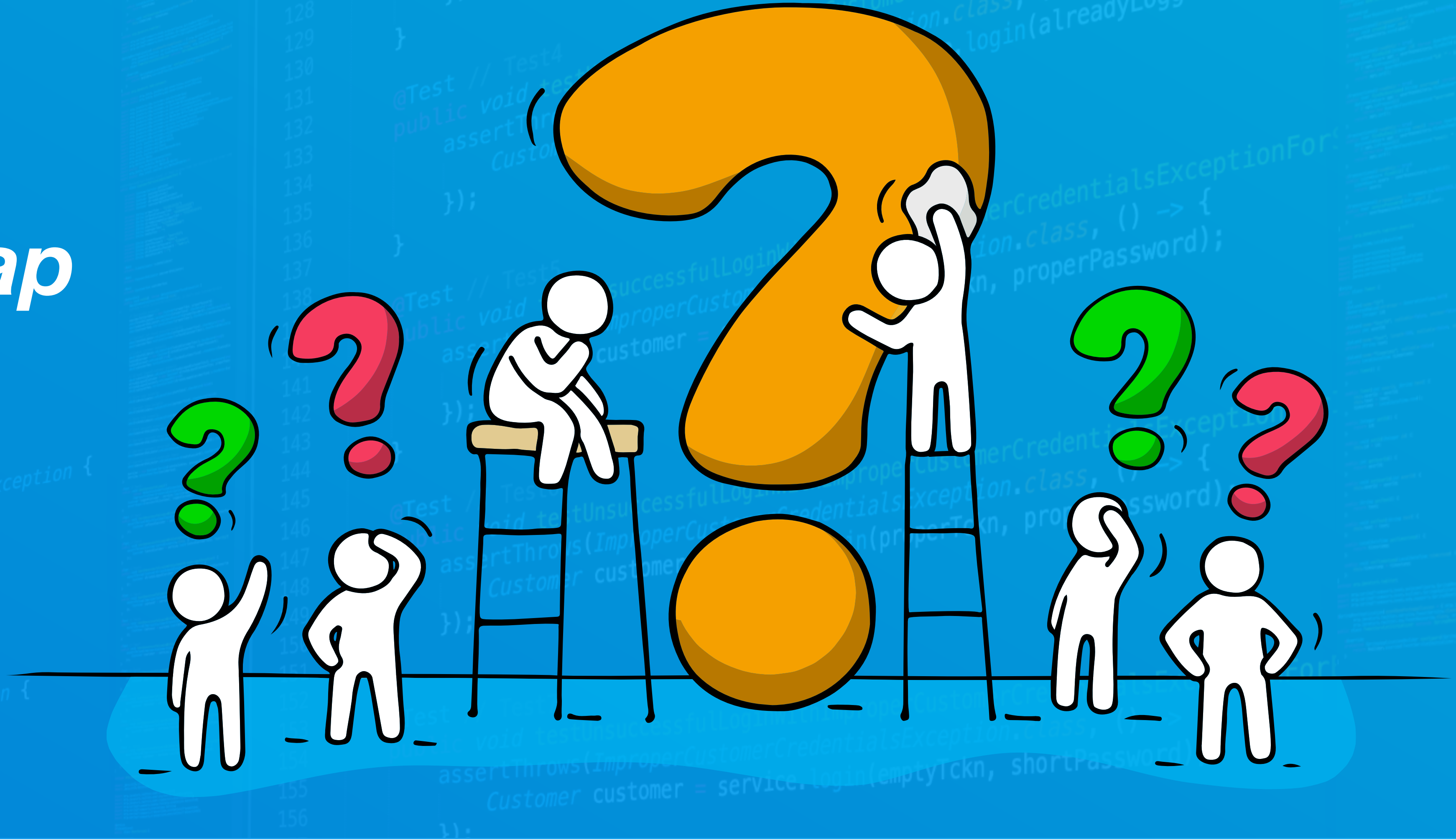
- **final** parametre sayesinde, geçilen argümanın değerinin, geçildiği metotta değiştirilmediğinden emin olunur
- Eğer böyle bir şeye ihtiyaç yoksa bu durumda parametreler **final** olarak tanıtılmaz ve bu durumda geçilen değerler metot içinde serbestçe değiştirilebilir

FinalParameter



- `constant.FinalParameter`

Soru ve Cevap Zamanı!



Ödevler





1. Bir önceki bölümde modellediğiniz köpek sınıfına davranışlar da ekleyin. Hangi davranışları eklemeyi düşünürdünüz? Bu davranışların arayüzünü ve gerçekleştirmesini belirleyip sınıfta kodlayın
 - a. Sonra test sınıfını yazıp **main** metodunda nesnelerini oluşturup değerler atayın



2. Köpeklerin sahiplerine hangi davranışları koymayı düşünürdünüz?

- a. Uygun metotları sınıfa ekleyin
- b. Köpeklerle sahipleri arasındaki ilişkiyi 1-1 düşünerek modelleyin
- c. Sonra test sınıfını yazıp **main** metodunda nesnelerini oluşturup değerler atayın, metotlarını çağırın



3. Bir önceki bölümde modellediğiniz iki boyutlu bir ortamda bir nokta soyutlamasına davranışlar da ekleyin. Örneğin, noktanın merkeze (origin) olan uzaklığı ile geçilen bir başka noktaya olan uzaklığını hesaplayıp döndüren metotları sınıfa ekleyin
 - a. Test sınıfının `main` metodunda, birisi merkezi (origin) temsil eden 3 nokta oluşturup üzerlerinde metot çağrılarını yapın



4. İlk bölümde oluşturduğunuz **Circle** (ya da **Daire**) sınıflarını son iki bölümde öğrendikleriniz ışığında tekrar ele alın, **Circle** sınıfındaki alanları ve metotları değerlendirin, gerekliyse yenilerini ekleyin
 - a. Daha sonra **main** metoda sahip olan ve bu metotta **Circle/Daire** nesneleri oluşturup üzerinde işlemler yapan **CircleTest/DaireTest** sınıfını oluşturun



5. Elektrikle çalışan bir lambayı ya da ampülü (bulb) modelleyin:
- a. Lamba açık mı kapalı mı, parlaklığı vb. özellikleri kullanarak uygun metotların imzalarını/arayüzlerini belirleyin ve gerçekleştirin
 - b. Sonra test sınıfını yazıp **main** metotta nesnelerini oluşturup üzerinde metot çağrıları yapın

Bundan Sonra?



Bundan Sonra?



- Bundan sonra kurucuları (constructors) inceleyeceğiz

Bölüm Sonu

Soru ve Cevap Zamanı!

